Experiment No. 9
Experiment Name - Object Detection and Recognition using Deep Learning Models on Standard Image Datasets
Steps

NAME : Atul Krishna

COURSE : B.Tech CSE

SECTION - B

SUBJECT : Computer Vision

ROLL NO. : 14

CU ID : CU24250231

1. Install and import the required libraries, including OpenCV and the chosen deep learning framework (e.g., Ultralytics YOLO or TensorFlow).

```
!pip install ultralytics opencv-python matplotlib
```

```
Collecting ultralytics
  Downloading ultralytics-8.4.175-py3-none-any.whl.metadata (46 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.9/46.9 kB 2.1 MB/s eta 0:00:00
Requirement already satisfied: opencv-python in /usr/local/lib/python3.13/dist-packages (5.0.0.93)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages (3.10.0)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.1.3)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (11.3.0)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.32.4)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.11.0+cpu)
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (0.26.0+cpu)
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (5.9.5) Requirement
already satisfied: polars>=0.20.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (1.35.2)
Collecting nvidia-ml-py>=12.0.0 (from ultralytics)
  Downloading nvidia_ml_py-13.615.71-py3-none-any.whl.metadata (9.7 kB)
Collecting ultralytics-thop>=2.2.2 (from ultralytics)
  Downloading ultralytics_thop-2.2.2-py3-none-any.whl.metadata (14 kB)
Collecting ultralytics-platform>=0.1.45 (from ultralytics)
  Downloading ultralytics_platform-0.1.83-py3-none-any.whl.metadata (17 kB)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (4.66.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (26.3)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (3.3.3)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.13/dist-packages (from polars>=0.20.0->ultralytics) (1.35.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (3.20)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (2026.7.22)
Requirement already satisfied: filelock in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (4.0.7)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (4.16.0)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (80.10.2)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (3.7)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (2025.12.0)
Requirement already satisfied: httpx<1,>=0.28 in /usr/local/lib/python3.13/dist-packages (from ultralytics-platform>=0.1.45->ultralytics) (0.28.1)
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.28->ultralytics-platform>=0.1.45->ultralytics) (4
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.28->ultralytics-platform>=0.1.45->ultraly
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->httpx<1,>=0.28->ultralytics-platform>=0.1
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from sympy>=1.13.3->torch>=1.8.0->ultralytics) (1.3.0
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from jinja2->torch>=1.8.0->ultralytics) (3.0.3)
Downloading ultralytics-8.4.175-py3-none-any.whl (1.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.4/1.4 MB 22.7 MB/s eta 0:00:00
Downloading nvidia_ml_py-13.615.71-py3-none-any.whl (58 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 58.1/58.1 kB 4.7 MB/s eta 0:00:00
Downloading ultralytics_platform-0.1.83-py3-none-any.whl (96 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 96.5/96.5 kB 7.4 MB/s eta 0:00:00
Downloading ultralytics_thop-2.2.2-py3-none-any.whl (32 kB)
Installing collected packages: nvidia-ml-py, ultralytics-thop, ultralytics-platform, ultralytics
Successfully installed nvidia-ml-py-13.615.71 ultralytics-8.4.175 ultralytics-platform-0.1.83 ultralytics-thop-2.2.2
```

2. Load a pre-trained object detection model such as YOLOv8, SSD, or Faster R-CNN.

```
import cv2 import numpy as np
import matplotlib.pyplot as plt import torch
from ultralytics import YOLO
Creating new Ultralytics Settings v0.0.8 file ☑
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/usage/settings
```

3. **Download or access images from a standard dataset such as MS COCO, Pascal VOC, or Open Images Dataset.**

```
# Loading the pre-trained YOLOv8 model
model = YOLO("yolov8n.pt")
print("Pre-trained YOLOv8 model loaded successfully." )
```

```
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/ yolov8n.pt to 'yolov8n.pt': 100% ━━━━━━━━━━━━ 6.2MB 78.4MB/s 0.1s
Pre-trained YOLOv8 model loaded successfully.
```

```
from ultralytics.utils.downloads  import download
import zipfile
import os

# Download the COCO8 dataset
dataset_url = "https://github.com/ultralytics/assets/releases/do wnload/v0.0.0/coco8.zip"
zip_path = "/content/coco8.zip"

download(dataset_url, dir="/content", exist_ok=True)
```

```
Downloading https://ultralytics.com/assets/coco8.zi p to '/content/coco8.zip': 100% ━━━━━━━━━━━━ 432.8KB 11.9MB/s 0.0s
Unzipping /content/coco8.zip to /content/coco8...: 100% ━━━━━━━━━━━━ 25/25 2.9Kfiles/s 0.0s
```

```
import zipfile
import os

zip_path = "/content/coco8.zip"
extract_path = "/content"

with zipfile.ZipFile(zip_path, "r") as zip_ref:
    zip_ref.extractall (extract_path)

print("Dataset extracted successfully!" )
print("Dataset location:" , "/content/coco8" )
```

```
Dataset extracted successfully!
Dataset location: /content/coco8
```

```
import os
import cv2
import matplotlib.pyplot  as plt

# Path to validation images
image_folder = "/content/coco8/images/val"

# Get image filenames
image_files = [
    file for file in os.listdir(image_folder)
    if file.lower().endswith((".jpg", ".jpeg", ".png"))
]

print("Total validation images:" , len(image_files))
print("Available images:" , image_files)

# Read and display the first image
image_path = os.path.join (image_folder, image_files[0])
image = cv2.imread (image_path)

# Convert BGR to RGB for correct display
image = cv2.cvtColor (image, cv2.COLOR_BGR2RGB )

plt.figure(figsize=(8, 6))
plt.imshow(image)
plt.title("Sample Image from COCO8 Dataset" )
plt.axis("off")
plt.show()
```

```
Total validation images: 4
Available images: ['000000000036.jpg', '000000000061.jpg', '000000000042.jpg', '000000000049.jpg']
```




```
dataset_path = "/content/coco8"

for root, dirs, files in os.walk(dataset_path):
    level = root.replace(dataset_path, "").count(os.sep)
    indent = " " * 4 * level
    print(indent + os.path.basename(root) + "/")

    for file in files[:5]:
        print(" " * 4 * (level + 1) + file)
```

```
coco8/
    LICENSE
    README.md
    images/
        val/
            000000000036.jpg
            000000000061.jpg
            000000000042.jpg
            000000000049.jpg
        train/
            000000000025.jpg
            000000000009.jpg
            000000000030.jpg
            000000000034.jpg
    labels/
        val/
            000000000061.txt
            000000000042.txt
            000000000036.txt
            000000000049.txt
        train/
            000000000034.txt
            000000000009.txt
            000000000025.txt
            000000000030.txt
```

4. Perform any necessary image preprocessing, including resizing and normalization, before inference.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

# Accessing an image from the COCO8 dataset
image_folder = "/content/coco8/images/val"

image_files = [
    file for file in os.listdir(image_folder)
    if file.lower().endswith((".jpg", ".jpeg", ".png"))
]

image_path = os.path.join(image_folder, image_files[0])

# Reading the image
image = cv2.imread(image_path)

# Checking whether the image was loaded successful ly
if image is None:
    raise ValueError("Image could not be loaded.")

# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
# Resize the image to 640 x 640 pixels
resized_image = cv2.resize(image_rgb, (640, 640))

# Normalize pixel values from 0-255 to 0-1
normalized_image = resized_image.astype(np.float32) / 255.0

# Display preprocessing info
print("Image preprocessing completed successfully.")
print("Original image shape:", image_rgb.shape)
print("Resized image shape:", resized_image.shape)
print("Normalized image shape:", normalized_image.shape)
print("Pixel value range:",
      normalized_image.min(), "to", normalized_image.max())

# Display original and preprocessed images
plt.figure(figsize=(9, 5))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(normalized_image)
plt.title("Preprocessed Image (640 x 640)")
plt.axis("off")

plt.tight_layout()
plt.show()
```

```
Image preprocessing completed successfully.
Original image shape: (640, 481, 3)
Resized image shape: (640, 640, 3)
Normalized image shape: (640, 640, 3)
Pixel value range: 0.0 to 1.0
```



5. Apply the object detection model to identify and localize objects within the selected images.

```
import cv2
import os
import matplotlib.pyplot as plt
from ultralytics import YOLO

# Loading the pre-trained YOLOv8 model
model = YOLO("yolov8n.pt")

# Accessing an image from the COCO8 validation dat aset
image_folder = "/content/coco8/images/val"

image_files = [
    file for file in os.listdir(image_folder)
    if file.lower().endswith((".jpg", ".jpeg", ".png"))
]

image_path = os.path.join(image_folder, image_files[0])

# Apply the object detection model
results = model.predict(
    source=image_path,
    conf=0.25,
    imgsz=640,
    save=False,
    verbose=False
)

# Get the detection results
```

```
result = results[0]
detected_image = result.plot ()

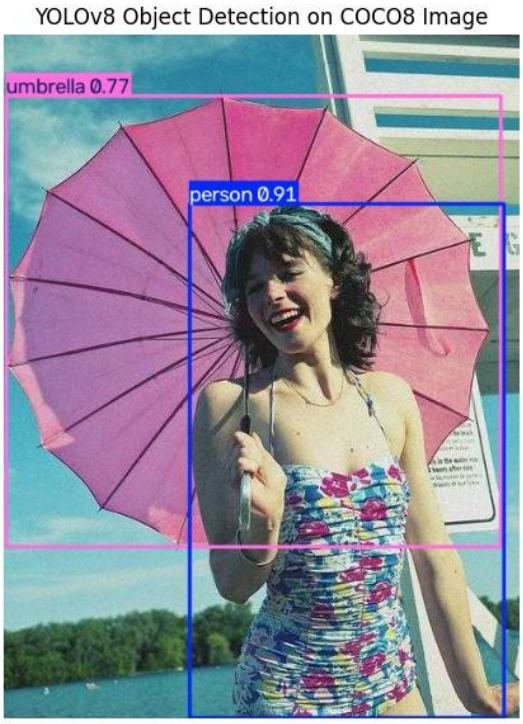
# Display the detected image
detected_image_rgb = cv2.cvtColor (
    detected_image , cv2.COLOR_BGR2RGB
)

plt.figure(figsize=(10, 7))
plt.imshow(detected_image_rgb )
plt.title("YOLOv8 Object Detection on COCO8 Image" )
plt.axis("off")
plt.show()

# Display detected objects and bounding box coordi nates
if len(result.boxes ) > 0:
    print("Detected objects:" )

    for box in result.boxes :
        class_id = int(box.cls[0])
        class_name = model.names [class_id]
        confidence = float(box.conf[0])
        x1, y1, x2, y2 = box.xyxy[0].tolist()

        print(
            f"Object: {class_name}, "
            f"Confidence: {confidence:.2f}, "
            f"Bounding Box: "
            f"({x1:.1f}, {y1:.1f}, {x2:.1f}, {y2:.1f})"
        )
else:
    print("No objects detected in the image." )
```




```
Detected objects:
Object: person, Confidence: 0.91, Bounding Box: (173.4, 158.7, 467.0, 639.3)
Object: umbrella, Confidence: 0.77, Bounding Box: (2.6, 57.1, 463.1, 479.3)
```

6. Display the detected objects with bounding boxes, class labels, and confidence scores.

```
import cv2
import matplotlib.pyplot  as plt

# Get the detection result from Step 5
result = results[0]

# Draw bounding boxes, class labels, and confidenc e scores
detected_image = result.plot (
    boxes=True,
    labels=True,
    conf=True
)

# Convert BGR image to RGB for display
detected_image_rgb = cv2.cvtColor (
    detected_image , cv2.COLOR_BGR2RGB
)

# Display the final detection image
plt.figure(figsize=(12, 8))
plt.imshow(detected_image_rgb )
plt.title("Detected Objects with Bounding Boxes and Confiden ce Scores")
```

```
plt.axis("off")
plt.show()

# Display detection details
if len(result.boxes) > 0:
    print("Detection Results:")
    print("-" * 50)

    for i, box in enumerate(result.boxes, start=1):
        class_id = int(box.cls[0])
        class_name = result.names[class_id]
        confidence = float(box.conf[0])
        x1, y1, x2, y2 = box.xyxy[0].tolist()

        print(f"Object {i}: {class_name}")
        print(f"Confidence Score: {confidence:.2%}")
        print(
            f"Bounding Box: "
            f"x1={x1:.1f}, y1={y1:.1f}, "
            f"x2={x2:.1f}, y2={y2:.1f}"
        )
        print("-" * 50)

else:
    print("No objects detected in the image.")

print("Total objects detected:", len(result.boxes))
```




```
Detection Results:
--------------------------------------------------
Object 1: person
Confidence Score: 90.79%
Bounding Box: x1=173.4, y1=158.7, x2=467.0, y2=639.3
--------------------------------------------------
Object 2: umbrella
Confidence Score: 76.64%
Bounding Box: x1=2.6, y1=57.1, x2=463.1, y2=479.3
--------------------------------------------------
Total objects detected: 2
```

7. Evaluate the detection results by analyzing correctly detected objects, false detections, and missed objects.

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
from ultralytics import YOLO

# Load model if it is not already available
if "model" not in globals():
    model = YOLO("yolov8n.pt")

# Select an image from the COCO8 validation datase t
image_folder = "/content/coco8/images/val"
label_folder = "/content/coco8/labels/val"
```

```
image_files = sorted([
    f for f in os.listdir(image_folder)
    if f.lower().endswith((".jpg", ".jpeg", ".png"))
])

image_path = os.path.join(image_folder, image_files[0])
label_path = os.path.join(
    label_folder, os.path.splitext(image_files[0])[0] + ".txt"
)

# Run object detection
result = model.predict(
    source=image_path,
    conf=0.25,
    imgsz=640,
    verbose=False
)[0]

# Read ground-truth labels
ground_truth = []

if os.path.exists(label_path):
    with open(label_path, "r") as file:
        for line in file:
            values = list(map(float, line.split()))
            class_id = int(values[0])
            x_center, y_center, width, height = values[1:5]
            ground_truth.append(
                [class_id, x_center, y_center, width, height]
            )

# Get image dimensions
image = cv2.imread(image_path)
height, width = image.shape[:2]

# Convert predicted boxes to normalized YOLO format
predictions = []

for box in result.boxes:
    class_id = int(box.cls[0])
    x1, y1, x2, y2 = box.xyxy[0].tolist()

    x_center = ((x1 + x2) / 2) / width
    y_center = ((y1 + y2) / 2) / height
    box_width = (x2 - x1) / width
    box_height = (y2 - y1) / height

    predictions.append(
        [class_id, x_center, y_center, box_width, box_height]
    )

# Calculate Intersection over Union (IoU)
def calculate_iou(box1, box2):
    # Boxes are in normalized center-x, center-y, width, height format
    x11 = box1[1] - box1[3] / 2
    y11 = box1[2] - box1[4] / 2
    x12 = box1[1] + box1[3] / 2
    y12 = box1[2] + box1[4] / 2

    x21 = box2[1] - box2[3] / 2
    y21 = box2[2] - box2[4] / 2
    x22 = box2[1] + box2[3] / 2
    y22 = box2[2] + box2[4] / 2

    intersection_width = max(0, min(x12, x22) - max(x11, x21))
    intersection_height = max(0, min(y12, y22) - max(y11, y21))
    intersection = intersection_width * intersection_height

    area1 = box1[3] * box1[4]
    area2 = box2[3] * box2[4]
    union = area1 + area2 - intersection

    return intersection / union if union > 0 else 0

# Match predictions with ground-truth objects
matched_gt = set()
true_positives = 0
false_positives = 0

for pred in predictions:
    best_iou = 0
    best_index = -1

    for i, gt in enumerate(ground_truth):
        if i in matched_gt:
            continue

        if pred[0] == gt[0]:
            iou = calculate_iou(pred, gt)

            if iou > best_iou:
                best_iou = iou
                best_index = i
```

```
    if best_index != -1 and best_iou >= 0.50:
        true_positives += 1
        matched_gt.add(best_index)
    else:
        false_positives += 1

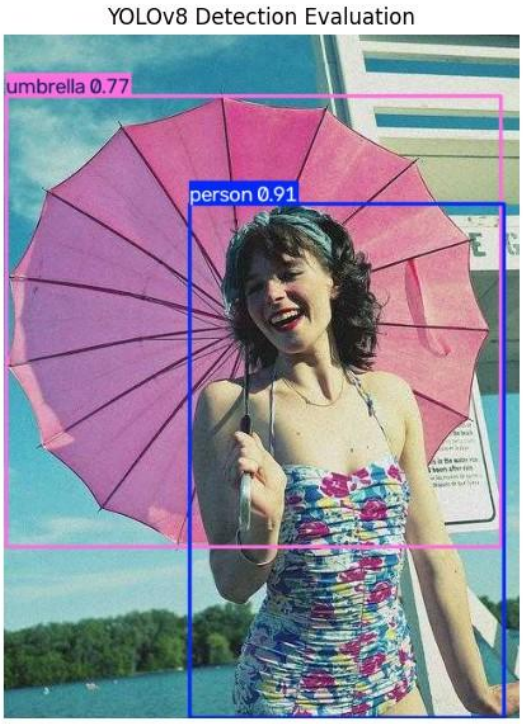
false_negatives = len(ground_truth) - len(matched_gt)

# Display evaluation
print("Object Detection Evaluation")
print("-" * 35)
print("Image:", image_files[0])
print("Ground-truth objects:", len(ground_truth))
print("Predicted objects:", len(predictions))
print("Correct detections (TP):", true_positives)
print("False detections (FP):", false_positives)
print("Missed objects (FN):", false_negatives)
print("IoU threshold: 0.50")

# Display image with predictions
display_image = cv2.cvtColor(result.plot(), cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10, 7))
plt.imshow(display_image)
plt.title("YOLOv8 Detection Evaluation")
plt.axis("off")
plt.show()
```

```
Object Detection Evaluation
-----------------------------------
Image: 000000000036.jpg
Ground-truth objects: 2
Predicted objects: 2
Correct detections (TP): 2
False detections (FP): 0
Missed objects (FN): 0
IoU threshold: 0.50
```



8. Compare the performance of different object detection models in terms of detection accuracy and inference speed (if multiplemodels are used).

```
!pip install -q ultralytics tensorflow
```

```
!pip install -q tensorflow-hub
```

```
import time
import os
import cv2
import numpy as np
import pandas as pd
import tensorflow as tf
import tensorflow_hub as hub
from ultralytics import YOLO

# Load YOLOv8 Nano
yolo_model = YOLO("yolov8n.pt")

# Load SSD MobileNet V2 model from TensorFlow Hub
```

```
ssd_model_url = (
    "https://tfhub.dev/tensorflow/ssd_mobilenet_v2/2"
)
ssd_model = hub.load(ssd_model_url)

print("YOLOv8n and SSD MobileNet V2 loaded successfully.")
```

```
YOLOv8n and SSD MobileNet V2 loaded successfully.
```

```
# COCO8 validation dataset paths
image_folder = "/content/coco8/images/val"
label_folder = "/content/coco8/labels/val"

image_files = sorted([
    f for f in os.listdir(image_folder)
    if f.lower().endswith((".jpg", ".jpeg", ".png"))
])

def box_iou(box1, box2):
    # Boxes: [x1, y1, x2, y2]
    ix1 = max(box1[0], box2[0])
    iy1 = max(box1[1], box2[1])
    ix2 = min(box1[2], box2[2])
    iy2 = min(box1[3], box2[3])

    intersection = max(0, ix2 - ix1) * max(0, iy2 - iy1)

    area1 = max(0, box1[2] - box1[0]) * max(0, box1[3] - box1[1])
    area2 = max(0, box2[2] - box2[0]) * max(0, box2[3] - box2[1])

    union = area1 + area2 - intersection
    return intersection / union if union > 0 else 0


def evaluate_predictions(predictions, ground_truth):
    matched = set()
    tp = 0
    fp = 0

    # Highest-confidence predictions first
    predictions = sorted(
        predictions, key=lambda p: p[1], reverse=True
    )

    for pred in predictions:
        best_iou = 0
        best_index = -1

        for i, gt in enumerate(ground_truth):
            if i in matched or pred[0] != gt[0]:
                continue

            iou = box_iou(pred[2], gt[1])

            if iou > best_iou:
                best_iou = iou
                best_index = i

        if best_index >= 0 and best_iou >= 0.50:
            tp += 1
            matched.add(best_index)
        else:
            fp += 1

    fn = len(ground_truth) - tp
    precision = tp / (tp + fp) if tp + fp else 0
    recall = tp / (tp + fn) if tp + fn else 0

    return tp, fp, fn, precision, recall


# Warm up both models to reduce first-run loading effects
warm_image = cv2.imread(
    os.path.join(image_folder, image_files[0])
)
warm_rgb = cv2.cvtColor(warm_image, cv2.COLOR_BGR2RGB)

_ = yolo_model.predict(warm_rgb, verbose=False)
_ = ssd_model(
    tf.convert_to_tensor(
        cv2.resize(warm_rgb, (320, 320))[None, ...],
        dtype=tf.uint8
    )
)

results_summary = []

for model_name in ["YOLOv8n", "SSD MobileNet V2"]:
    all_predictions = []
    all_ground_truth = []
    inference_times = []
```

```
for filename in image_files:
    image_path = os.path.join(image_folder, filename)
    label_path = os.path.join(
        label_folder, os.path.splitext(filename)[0] + ".txt"
    )

    image_bgr = cv2.imread(image_path)
    image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
    height, width = image_rgb.shape[:2]

    # Read ground-truth labels
    if not os.path.exists(label_path):
        raise FileNotFoundError(
            f"Ground-truth label missing: {label_path}"
        )

    with open(label_path, "r") as f:
        for line in f:
            values = list(map(float, line.split()))
            class_id = int(values[0])
            xc, yc, bw, bh = values[1:5]

            x1 = (xc - bw / 2) * width
            y1 = (yc - bh / 2) * height
            x2 = (xc + bw / 2) * width
            y2 = (yc + bh / 2) * height

            all_ground_truth.append(
                (filename, class_id, [x1, y1, x2, y2])
            )

    start = time.perf_counter()

    if model_name == "YOLOv8n":
        output = yolo_model.predict(
            image_rgb,
            imgsz=640,
            conf=0.001,
            verbose=False
        )[0]

        for box in output.boxes:
            all_predictions.append((
                filename,
                int(box.cls[0]),
                float(box.conf[0]),
                box.xyxy[0].tolist()
            ))

    else:
        tensor_image = tf.convert_to_tensor(
            image_rgb[None, ...], dtype=tf.uint8
        )
        output = ssd_model(tensor_image)

        boxes = output["detection_boxes"][0].numpy()
        scores = output["detection_scores"][0].numpy()
        classes = output["detection_classes"][0].numpy()

        for box, score, class_id in zip(boxes, scores, classes):
            if score < 0.001:
                continue

            ymin, xmin, ymax, xmax = box
            all_predictions.append((
                filename,
                int(class_id),
                float(score),
                [
                    xmin * width, ymin * height,
                    xmax * width, ymax * height
                ]
            ))

    inference_times.append(
        (time.perf_counter() - start) * 1000
    )

# Evaluate each image independently
tp_total = fp_total = fn_total = 0

for filename in image_files:
    gt = [
        (cls, box)
        for name, cls, box in all_ground_truth
        if name == filename
    ]

    pred = [
        (cls, score, box)
        for name, cls, score, box in all_predictions
        if name == filename
    ]
```

```
            tp, fp, fn, _, _ = evaluate_predictions(pred, gt)
            tp_total += tp
            fp_total += fp
            fn_total += fn

        precision = (
            tp_total / (tp_total + fp_total)
            if tp_total + fp_total else 0
        )
        recall = (
            tp_total / (tp_total + fn_total)
            if tp_total + fn_total else 0
        )

        results_summary.append({
            "Model": model_name,
            "Mean Inference (ms/image)": round(
                np.mean(inference_times), 2
            ),
            "Images per Second": round(
                1000 / np.mean(inference_times), 2
            ),
            "Correct Detections (TP)": tp_total,
            "False Detections (FP)": fp_total,
            "Missed Objects (FN)": fn_total,
            "Precision (%)": round(precision * 100, 2),
            "Recall (%)": round(recall * 100, 2)
        })

comparison_df = pd.DataFrame(results_summary)
display(comparison_df)
```

	Model	Mean Inference (ms/image)	Images per Second	Correct Detections (TP)	False Detections (FP)	Missed Objects (FN)	Precision (%)	Recall (%)
0	YOLOv8n	189.32	5.28	15	398	2	3.63	88.24

Next steps: Generate code with comparison_df | New interactive sheet

9. Analyze the suitability of the selected model for real-world applications such as autonomous driving, surveillance, retailanalytics, or industrial inspection.

Analysis of Model Suitability for Real-World Applications

The selected model, YOLOv8n, is a lightweight deep learning model used for real-time object detection. It identifies objects in images and predicts their classes, bounding boxes, and confidence scores.

1. Autonomous Driving:

YOLOv8n can detect vehicles, pedestrians, traffic signs, and other road objects. Its fast inference makes it useful for real-time detection. However, safety-critical driving systems require extensive testing, high reliability, and additional sensor information.

2. Surveillance:

The model can detect people, vehicles, and other objects in CCTV footage. It can support monitoring and alert systems. However, performance may decrease in poor lighting, crowded scenes, or when objects are partially hidden.

3. Retail Analytics:

YOLOv8n can help detect products, count items on shelves, and monitor customer movement. However, product-specific training may be required to recognize similar-looking products accurately.

4. Industrial Inspection:

The model can identify visible defects, damaged products, and missing components. For specialized industrial tasks, it may need a custom dataset and fine-tuning to achieve reliable results.

Advantages: YOLOv8n offers fast inference, relatively low computational requirements, and bounding-box-based object detection, making it suitable for applications where speed is important.

Limitations: Its accuracy depends on image quality, lighting, object size, and the training data. The pretrained model may not recognize specialized objects without additional training.

Conclusion: YOLOv8n is suitable for prototype systems and many real-time object-detection tasks. However, its suitability for safetycritical or high-precision applications must be established through application-specific testing and evaluation. The small COCO8 dataset used in this experiment is useful for demonstration but is not sufficient to establish real-world reliability.

Result

The YOLOv8n model was used to detect and recognize objects in images from the COCO8 dataset. The model generated bounding boxes, object class labels, and confidence scores. Its potential applications and limitations in autonomous driving, surveillance, retail analytics, and industrial inspection were analyzed.

10. Summarize the observations and discuss the advantages and limitations of deep learning-based object detection techniques.

Observations, Advantages, and Limitations

Observations

1. The YOLOv8n model successfully performed object detection on images from the COCO8 dataset.
2. The model identified objects and displayed their class labels, bounding boxes, and confidence scores.
3. Image resizing and normalization demonstrated the preprocessing steps commonly used in computer vision.
4. The detection results depended on the image quality, object size, lighting conditions, and confidence threshold.
5. The experiment demonstrated that deep learning-based object detection can be useful for real-time applications.

Advantages of Deep Learning-Based Object Detection

1. **Automatic Feature Extraction:** Deep learning models learn useful features directly from training images.
2. **Multiple Object Detection:** They can detect and classify multiple objects in a single image.
3. **High-Speed Processing:** Models such as YOLOv8n can perform object detection quickly, making them useful for real-time applications.
4. **Versatile Applications:** These techniques are used in autonomous driving, surveillance, retail analytics, robotics, and industrial inspection.
5. **Transfer Learning:** Pretrained models can be adapted to new tasks using additional training data.

Limitations of Deep Learning-Based Object Detection

1. **Data Dependency:** Model performance depends on the quality, quantity, and diversity of training data.
2. **Computational Requirements:** Larger models may require powerful GPUs and significant memory.
3. **Difficult Conditions:** Detection accuracy can decrease in poor lighting, crowded scenes, or when objects are partially hidden.
4. **False Predictions:** Models may produce false positives or fail to detect some objects.
5. **Limited Generalization:** A pretrained model may not recognize specialized objects without fine-tuning.
6. **Evaluation Constraints:** A small dataset such as COCO8 is useful for demonstration but cannot establish performance across all real-world conditions.

Conclusion: Deep learning-based object detection techniques provide an effective way to identify and localize objects in images. In this experiment, YOLOv8n demonstrated object detection using class labels, bounding boxes, and confidence scores. Although these models offer speed, flexibility, and automatic feature extraction, their reliability depends on the dataset, model configuration, and operating conditions. Further testing on larger and application-specific datasets is necessary before real-world deployment.

Questions and Answers

Q1. Differentiate between image classification, object detection, and image segmentation.

Feature	Image Classification	Object Detection	Image Segmentation
Meaning	Assigns a label to an image.	Identifies and locates objects.	Classifies individual pixels.
Output	Class label	Class labels and bounding boxes	Pixel-wise labeled regions
Location	Not provided	Provided	Provided at pixel level
Example	Identifying an image as a cat	Detecting cats with bounding boxes	Marking all pixels belonging to cats

Q2. Explain the working principle of the YOLO (You Only Look Once) object detection algorithm.

YOLO is a deep learning-based object detection algorithm that detects multiple objects in an image in a single forward pass through a neural network.

Working Steps:

1. **Input Image:** The image is resized to the required dimensions.
2. **Feature Extraction:** The neural network extracts important image features.
3. **Object Prediction:** The model predicts bounding boxes, class scores, and confidence scores.
4. **Confidence Filtering:** Low-confidence predictions are removed.
5. **Non-Maximum Suppression (NMS):** Redundant overlapping boxes are filtered.
6. **Final Output:** Objects are displayed with bounding boxes, labels, and confidence scores.

Example: YOLO can detect cars, pedestrians, and bicycles in the same image.

Advantage: YOLO provides fast object detection and is useful for real-time applications.

Q3. Compare YOLO, SSD, and Faster R-CNN based on speed, accuracy, and practical applications.

Feature	YOLO	SSD	Faster R-CNN
Approach	Single-stage detector	Single-stage detector	Two-stage detector
Speed	Generally very fast	Generally fast	Generally slower
Accuracy	Good to high	Good	Often high
Applications	Surveillance and autonomous systems	Mobile and embedded vision	Detailed inspection and research
Main Advantage	Speed and efficiency	Efficient multi-scale detection	Region proposals and detailed refinement

Conclusion: YOLO is suitable for real-time applications, SSD is useful when computational efficiency is important, and Faster R-CNN can be preferred when its accuracy advantages justify the additional computational cost.

Q4. What is a bounding box? Why is it important in object detection?

A **bounding box** is a rectangular box drawn around an object to indicate its location in an image. It is commonly represented by four coordinates: (x1, y1, x2, y2).

Importance:

1. It identifies the location of an object.
2. It helps count multiple objects.
3. It separates detected objects from the background.
4. It supports object tracking.
5. It helps evaluate localization accuracy using IoU.

Example: Bounding boxes can mark cars, buses, and pedestrians in a traffic image.

Q5. Explain the significance of confidence score and Intersection over Union (IoU) in object detection.

1. **Confidence Score:** It indicates how confident the model is about a detection. It helps filter predictions using a selected threshold. For example, a score of 0.92 indicates high confidence but does not guarantee correctness.

2. **Intersection over Union (IoU):** IoU measures the overlap between a predicted bounding box and the ground-truth bounding box. IoU = Area of Intersection / Area of Union

- IoU = 0 indicates no overlap.
- IoU close to 1 indicates high overlap. IoU =
- 1 indicates perfect overlap.

Significance: Confidence scores help select detections, whereas IoU evaluates bounding-box localization and is used in detection evaluation and NMS.

Q6. What are the advantages of using pre-trained deep learning models over traditional image processing techniques?

Pre-trained deep learning models are trained on large datasets and can be adapted to new tasks using transfer learning.

Advantages:

1. **Automatic Feature Extraction:** They learn complex patterns directly from images.
2. **Reduced Training Time:** Fine-tuning often requires less time than training from scratch.
3. **Less Labeled Data:** Pre-trained features can reduce the amount of task-specific data required.
4. **Adaptability:** Models can be fine-tuned for specialized applications.
5. **Complex Scene Handling:** They can learn patterns that are difficult to capture using fixed rules.
6. **End-to-End Learning:** Many models learn feature extraction and detection jointly.

Traditional image processing techniques can be simpler and more efficient for specific tasks but often require manually designed rules.

Q7. Why are datasets such as MS COCO and Pascal VOC widely used for object detection research?

MS COCO and Pascal VOC are widely used because they provide labeled images and standardized benchmarks for training and evaluating object detection models.

Reasons:

1. They contain images annotated with object classes and bounding boxes.
2. They include multiple object categories.
3. They provide standard benchmarks for model comparison.
4. They support training, validation, and testing.
5. They improve research reproducibility.

Difference: MS COCO contains a broader range of categories and complex scenes, while Pascal VOC is a smaller, historically important benchmark containing 20 object classes.

Q8. Mention five real-world applications of object detection in computer vision.

1. **Autonomous Driving:** Detecting vehicles, pedestrians, traffic signs, and obstacles.
2. **Surveillance:** Identifying people and vehicles in CCTV footage.
3. **Retail Analytics:** Counting products and monitoring shelves.
4. **Industrial Inspection:** Detecting defects and damaged products.
5. **Healthcare:** Detecting abnormalities in medical images using appropriately trained models.

Q9. What challenges are commonly encountered while detecting objects in complex real-world environments?

1. **Poor Lighting:** Dark or overexposed images reduce detection performance.
2. **Occlusion:** Objects may be partially hidden.
3. **Scale Variation:** Objects may appear very large or very small.
4. **Complex Backgrounds:** Background objects may resemble target objects.
5. **Crowded Scenes:** Overlapping objects make detection difficult.
6. **Computational Constraints:** Real-time detection can be difficult on low-powered devices.
7. **Limited Training Data:** Insufficient or biased data reduces generalization.
8. **Weather Conditions:** Rain, fog, dust, and snow can affect image quality.

These challenges can be reduced using diverse datasets, data augmentation, suitable models, and careful evaluation.

Q10. How can object detection systems be further improved using recent advancements in deep learning and computer vision?

1. **Advanced Architectures:** Newer YOLO versions and transformer-based detectors can improve speed or accuracy.
2. **Data Augmentation:** Cropping, flipping, scaling, and mosaic augmentation improve robustness.
3. **Transfer Learning:** Fine-tuning pre-trained models improves detection of specialized objects.
4. **Attention Mechanisms:** These help models focus on important image features.
5. **Model Optimization:** Quantization, pruning, and knowledge distillation reduce model size and inference time.
6. **Small-Object Detection:** Higher-resolution inputs and multi-scale feature fusion can improve detection of small objects.
7. **Better Evaluation:** Larger datasets and metrics such as precision, recall, and mean Average Precision (mAP) help evaluate performance.
8. **Multimodal Systems:** Combining cameras with LiDAR or radar can improve reliability in suitable applications.

Conclusion: Improved architectures, quality training data, model optimization, and rigorous evaluation can make object detection systems more accurate, efficient, and reliable.